Northern University of Business and Technology Khulna
Department of Computer Science and Engineering

TEAM CONTRIBUTION & CODE GUIDE

Project Title: AI-Powered Resume Analyzer and Career Copilot (RESUMIND)

Course Title: Mobile Computing Lab

Course Code: CSE4204

Submitted to:
Md. Riaz Mahmud
Assistant Professor
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

Submission Date: 22 October 2026

TEAM INFORMATION SHEET

Field	Details
Official Team Name	CSE4204-8A-T05
Section	8A
Project Title	AI-Powered Resume Analyzer and Career Copilot (RESUMIND)
Team Leader	Mahammad Hasan (ID: 11220320828)
Team Member List with IDs	1. Mahammad Hasan (ID: 11220320828) - Team Leader / Frontend Developer + UI/UX Designer 2. Shanawaz Sakib (ID: 11220320916) - Core System Integration Engineer 3. Ashikur Rahman Himel (ID: 11220320833) - Database Storage & Quality Controller 4. Rasel Ratul (ID: 11220320827) - Backend Developer
Deployment Link (Live URL)	https://ai-resume-analyzer-psi-lime.vercel.app
GitHub Repository Link	https://github.com/mahamud894/AI_Resume_Analyzer
Figma Link	https://www.figma.com/design/Ls0B3DIjxRKrP2RIzYQxPG/AI-Resume-Analyzer?node-id=2-2&t=lFEVQCoUHt0JSumI-1

1. Purpose of This Guide

This guide shows **who did what** in RESUMIND, **which technologies** each member used, and **which file contains which part** of the system. It is written so that during the presentation or viva, if the teacher asks us to explain or change something, any member can open the right file immediately and make the change correctly.

- Chapter 2 gives the big picture: folders, how a request travels, how to run the project.
- Chapter 3 shows the responsibility map of the team.
- Chapters 4–7 have one chapter per member: responsibilities, technologies, files, how to explain it and the most likely change requests.
- Chapter 8 is a quick lookup table of common changes; Chapter 9 explains how to make and test a change safely.

2. Project at a Glance

2.1 Folder structure

```
AI_Resume_Analyzer/
├── frontend/                 React 19 + React Router 8 app (deployed on Vercel)
│   ├── app/routes.ts         list of all pages (URLs)
│   ├── app/routes/           one file per page
│   ├── app/components/       reusable UI pieces
│   ├── app/lib/              API client, auth store, helpers
│   ├── app/app.css           colours, fonts, global styles
│   └── types/index.d.ts      TypeScript types shared by the pages
├── backend/                  Node.js + Express 5 REST API (deployed on Render)
│   ├── server.js             starts the server
│   ├── src/app.js            middleware + routes
│   ├── src/routes/           URL → controller wiring
│   ├── src/controllers/      what each endpoint does
│   ├── src/services/         Gemini, Europass, cover letter, PDF, GridFS
│   ├── src/models/           MongoDB collections (Mongoose)
│   ├── src/middleware/       JWT check, validation, upload, errors
│   └── tests/                48 automated API tests
├── database/                 schema docs, SQL equivalent, seed data
├── .github/workflows/ci.yml  tests on every push
├── render.yaml               backend deployment settings
└── docker-compose.yml        run everything locally with Docker
```

2.2 How one request travels (example: Analyze Resume)

1. **Frontend page** `routes/upload.tsx` collects the PDF and job details and calls `resumeApi.upload()` and `resumeApi.analyze()` in `lib/api.ts`.

2. **Route** `backend/src/routes/resumeRoutes.js` matches `POST /api/resume/analyze` and runs `requireAuth → aiLimiter → analyzeRules`.
3. **Controller** `resumeController.analyze()` loads the resume, calls `extractPdfText()` (pdfService) and `analyzeResume()` (geminiService).
4. **Service** `geminiService.generateJson()` sends the prompt and JSON schema to Gemini; `normalizeFeedback()` cleans the answer.
5. **Model** `Analysis` is saved in MongoDB; the controller returns `201 { analysis }`.
6. **Frontend** navigates to `routes/resume.tsx`, which shows the scores with `Summary`, `ATS` and `Details` components.

2.3 Run the project locally

```
# terminal 1 — backend (needs backend/.env)
cd backend && npm install && npm run dev        # http://localhost:5000

# terminal 2 — frontend (needs frontend/.env)
cd frontend && npm install && npm run dev       # http://localhost:5173

# after any backend change
cd backend && npm test                          # 48 tests must pass
# after any frontend change
cd frontend && npm run typecheck && npm run build
```

3. Team Responsibility Map

Table 1: Who owns which part

Member	Role	Main area	Main folders
Mahammad Hasan 11220320828	Team Leader / Frontend Developer + UI/UX Designer	All pages, components, design, auth state on the client	frontend/app/
Shanawaz Sakib 11220320916	Core System Integration Engineer	Gemini AI, Europass rewrite, cover letter, end-to-end flow	backend/src/services/ (AI), improve & cover-letter controllers
Ashikur Rahman Himel 11220320833	Database Storage & Quality Controller	MongoDB models, GridFS, validation, tests, CI	backend/src/models/, database/, backend/tests/
Rasel Ratul 11220320827	Backend Developer	Express API, authentication, security, deployment	backend/src/ (app, routes, auth, middleware), render.yaml

Tip: Everyone should be able to explain the whole flow (Chapter 2.2), but each member answers detailed questions about their own chapter.

4. Mahammad Hasan — Team Leader / Frontend Developer + UI/UX Designer

4.1 Responsibilities

- Team coordination, weekly planning and the GitHub workflow.
- Overall app structure with React Router 8 and protected routes.
- All pages: landing, login/sign up, dashboard, upload, result, Europass CV editor, cover letter, account, wipe.
- RESUMIND visual design with Tailwind CSS 4, responsive layout for phone, tablet and desktop.
- Client-side auth state (Zustand) and the API client that attaches the JWT.
- Result features: score history chart, before/after view, printable report PDF, 404 page.

Table 2: Technologies used

Technology	Used for
React 19 + TypeScript	Components and pages with type safety
React Router 8	Page routing (`app/routes.ts`), page titles (`meta`), SPA build for Vercel
Tailwind CSS 4	All styling; theme colours in `app.css` (`@theme`)
Zustand	Global login state (`lib/auth.ts`)
pdf.js (pdfjs-dist)	Creates the page-1 preview image of the uploaded PDF in the browser
Fetch API	Calls to the backend in `lib/api.ts`
@react-oauth/google	"Sign in with Google" button on the auth card

4.2 Files

Table 3: Files and what they contain

File	What it does	Key functions / parts
`frontend/app/root.tsx`	HTML shell, fonts, global error page (404)	Layout, ErrorBoundary
`frontend/app/routes.ts`	List of every page and its URL	index, route(...)
`frontend/app/routes/landing.tsx`	Public home page with login popup	Landing, LandingNav, AuthModal

File	What it does	Key functions / parts
`frontend/app/routes/auth.tsx`	Full login / sign-up page	Auth
`frontend/app/routes/home.tsx`	Dashboard with all resumes	Home
`frontend/app/routes/upload.tsx`	Upload form + progress steps	Upload
`frontend/app/routes/resume.tsx`	Analysis result page	Resume
`frontend/app/routes/improved.tsx`	Europass CV editor, before/after score	Improved
`frontend/app/routes/cover-letter.tsx`	Cover letter form, editor, PDF	CoverLetterPage
`frontend/app/routes/account.tsx`	Change password, delete account	Account
`frontend/app/routes/wipe.tsx`	Delete all my data	WipeApp
`frontend/app/components/LandingSections.tsx`	How it works, features, team, footer	HowItWorks, Features, Team, TEAM
`frontend/app/components/AuthCard.tsx`	Login / sign-up form and Google button	AuthCard
`frontend/app/components/Navbar.tsx`	Top navigation and logout	Navbar
`frontend/app/components/PageChrome.tsx`	Shared top bar, title, card	TopBar, PageTitle, Card
`frontend/app/components/FileUploader.tsx`	Drag-and-drop PDF box	FileUploader
`frontend/app/components/JobFields.tsx`	Company / job title / description inputs	JobFields, readJobDetails
`frontend/app/components/Summary.tsx, ATS.tsx, Details.tsx`	Score overview, ATS card, tips per category	Summary, ATS, Details
`frontend/app/components/ScoreGauge.tsx, ScoreCircle.tsx, ScoreBadge.tsx`	Score visuals and colour badges	ScoreGauge, ScoreCircle

File	What it does	Key functions / parts
`frontend/app/components/EuropassCv.tsx`	Editable A4 Europass CV	EuropassCv
`frontend/app/components/ImprovementReport.tsx`	List of AI changes	ImprovementReport
`frontend/app/components/BeforeAfter.tsx`	Before/after score comparison	BeforeAfter
`frontend/app/components/ScoreHistoryChart.tsx`	Score history chart	ScoreHistoryChart
`frontend/app/components/CoverLetterSheet.tsx`	A4 letter view	CoverLetterSheet
`frontend/app/components/ReportPrint.tsx`	Printable analysis report	ReportPrint
`frontend/app/components/ProgressSteps.tsx`	Step-by-step progress while AI works	ProgressSteps
`frontend/app/components/ScrollEffects.tsx`	Scroll reveal animations	ScrollEffects
`frontend/app/lib/api.ts`	All backend calls + JWT header + 401 handling	authApi, resumeApi, tokenStorage
`frontend/app/lib/auth.ts`	Login state (Zustand store)	useAuthStore
`frontend/app/lib/hooks.ts`	Route guard, protected file loader	useRequireAuth, useProtectedFile
`frontend/app/lib/pdf2img.ts`	PDF page 1 → PNG preview	convertPdfToImage
`frontend/app/lib/europass.ts`, `coverLetter.ts`	Read edits from the CV, letter → text	collectCvEdits, letterToText
`frontend/app/constants.ts`	API URL, max file size, allowed types	API, MAX_FILE_SIZE
`frontend/app/app.css`	Theme colours, fonts, animations, print styles	@theme
`frontend/types/index.d.ts`	Types for Resume, Analysis, Feedback, CV	interfaces

4.3 How to explain it

"The frontend is a React single-page app. `routes.ts` lists the pages; each page is a file in `routes/`. Every backend call goes through `lib/api.ts`, which adds the JWT from local storage. `useRequireAuth()` protects private pages. The design uses Tailwind CSS with our colours in `app.css`, and every layout is mobile-first."

Table 4: If sir asks for a change — where to edit

Change request	Edit here	Then check
Change a colour / font	`frontend/app/app.css` → `@theme` variables	Landing, dashboard, result
Change landing text or team info	`components/LandingSections.tsx` (`STEPS`, `TEAM`), hero in `routes/landing.tsx`	Landing page
Change a button label or page title	the page file in `routes/`; title in its `meta()`	That page
Add a new page	add `route('/x', 'routes/x.tsx')` in `routes.ts`, create the file, call `useRequireAuth()` if private	Navigate to /x
Change score colour levels (green/amber/red)	`ScoreBadge.tsx`, `Summary.tsx`, `ATS.tsx`, `Details.tsx` (`score > 70`, `> 49` …)	Result page
Change max upload size in the UI	`constants.ts` → `MAX_FILE_SIZE` (keep equal to backend)	Upload page
Password rule in the form	`AuthCard.tsx` (`password.length < 8`, `minLength`), `account.tsx`	Sign up, Account

5. Shanawaz Sakib — Core System Integration Engineer

5.1 Responsibilities

- Google Gemini integration: prompts and JSON schemas for analysis, Europass rewrite, before/after score and cover letter.
- Retry logic for temporary AI errors and clear error messages.
- End-to-end flow: PDF → text → Gemini → saved feedback → UI.
- Europass post-check rules that stop invented numbers and wrong dates.
- AI rate limit (15 requests per user per hour) and the Docker Compose setup.
- Week 08 AI response mapper and cache prototype; frontend–backend connection on Vercel.

Table 5: Technologies used

Technology	Used for
Google Gemini (@google/genai)	All AI features, JSON mode with `responseJsonSchema`
pdf-parse	Extracts text from the uploaded PDF
JSON Schema	Forces the AI answer into a fixed structure
express-rate-limit	AI quota per user (`aiLimiter`)
Docker / Docker Compose	Run frontend + backend together locally

5.2 Files

Table 6: Files and what they contain

File	What it does	Key functions / parts
`backend/src/services/geminiService.js`	Shared Gemini call with retries; analysis prompt, schema and cleaning	generateJson, buildPrompt, FEEDBACK_SCHEMA, normalizeFeedback, analyzeResume
`backend/src/services/europassService.js`	Europass CV schema, rewrite prompt, post-check rules	EUROPASS_SCHEMA, buildImprovePrompt, normalizeEuropassCv, postCheck, normalizeDate, expandQualification, eqfLevelFor
`backend/src/services/coverLetterService.js`	Cover letter prompt, schema, fixed header/salutation, word count	COVER_LETTER_SCHEMA, buildCoverLetterPrompt, assembleLetter, generateCoverLetter, TONES
`backend/src/services/pdfService.js`	PDF bytes → plain text	extractPdfText
`backend/src/controllers/improveController.js`	Europass endpoints: generate, get, save edits, before/after score	improve, getImproved, saveImprovedEdit, scoreImproved
`backend/src/controllers/coverLetterController.js`	Cover letter endpoints	createCoverLetter, getCoverLetter, saveCoverLetterEdit

File	What it does	Key functions / parts
backend/src/routes/resumeRoutes.js (aiLimiter)	AI quota: 15 per hour per user	aiLimiter
docker-compose.yml	Starts backend (5000) and frontend (3000)	services
frontend/utils/aiMapper.js, cache.js	Week 08 prototype: map AI response, cache results	mapAIResponse, getAnalysisWithCache
frontend/vercel.json, app/constants.ts	Frontend deployment config, backend URL	API

5.3 How to explain it

"All AI features use one function, `generateJson()` in `geminiService.js`. It sends a prompt and a JSON schema to Gemini in JSON mode, so the answer always has our structure. If Gemini is busy (500/503/504) it retries after 2 and 4 seconds. The answer is cleaned in code — for example `normalizeFeedback()` clamps scores to 0–100 and `postCheck()` replaces invented numbers in the Europass CV with placeholders."

Table 7: If sir asks for a change — where to edit

Change request	Edit here	Then check
Change how the AI scores / the analysis prompt	`geminiService.js` → `buildPrompt()`	Analyze a resume
Add or rename a score category	`FEEDBACK_SCHEMA` + `normalizeFeedback()` in geminiService.js, `models/Analysis.js`, `types/index.d.ts`, `Summary.tsx`/`Details.tsx`	Tests + result page
Use another Gemini model	`GEMINI_MODEL` in backend `.env` / Render environment	Analyze
Change number of retries / wait time	`MAX_ATTEMPTS` and `sleep(2000 * attempt)` in geminiService.js	—
Allow longer resumes	`MAX_RESUME_CHARS` (30 000) in geminiService.js	Analyze
Change Europass rules	`buildImprovePrompt()` and `postCheck()` in europassService.js	europass.test.js
Change cover letter length	`MIN_WORDS` / `MAX_WORDS` in coverLetterService.js	Generate letter

Change request	Edit here	Then check
Add a new letter tone	`TONES` in coverLetterService.js, `coverLetterRules` in validate.js, tone buttons in `cover-letter.tsx`	Generate letter
Change AI requests per hour	`aiLimiter → limit: 15` in resumeRoutes.js	—

6. Ashikur Rahman Himel — Database Storage & Quality Controller

6.1 Responsibilities

- Database design following the ERD: five Mongoose models with references and unique indexes.
- File storage in MongoDB GridFS (PDFs and preview images).
- Schema documentation (`mongodb-schema.md`), the SQL equivalent (`schema.sql`) and seed data.
- Input validation rules for every endpoint.
- 48 automated API tests (Vitest + Supertest + in-memory MongoDB) and the GitHub Actions CI.
- Dependency security audit and the Week 10 testing report.

Table 8: Technologies used

Technology	Used for
MongoDB Atlas	Cloud database (free M0 cluster)
Mongoose 9	Schemas, validation, indexes, references
GridFS	Stores PDF and image bytes in 255 KB chunks
express-validator	Input rules for every request
Vitest + Supertest	Automated HTTP tests of the API
mongodb-memory-server	Temporary database for tests
GitHub Actions	Runs tests and frontend build on every push

6.2 Files

Table 9: Files and what they contain

File	What it does	Key functions / parts
`backend/src/models/User.js`	users collection	username, email (unique), passwordHash, hasPassword, googleId, passwordChangedAt
`backend/src/models/Resume.js`	resumes collection	userId, fileId, imageId, originalName, fileSize
`backend/src/models/Analysis.js`	analyses + embedded feedback	tipSchema, categorySchema, feedbackSchema
`backend/src/models/ImprovedResume.js`	Europass CV, edits, changes, after-score	analysisId (unique), europassCv, editedCv, afterFeedback
`backend/src/models/CoverLetter.js`	Cover letter + edited text	improvedResumeId (unique), MAX_EDITED_TEXT
`backend/src/services/fileStorage.js`	Save / read / delete files in GridFS	saveFile, readFile, sendFile, deleteFiles, isRealPdf
`backend/src/config/db.js`	Connect / disconnect MongoDB	connectDB, disconnectDB
`backend/src/middleware/validate.js`	All input rules	registerRules, loginRules, analyzeRules, coverLetterRules, idParamRules …
`database/mongodb-schema.md, schema.sql`	Database documentation + SQL version of the ERD	—
`database/seed.js, sample-data.json`	Demo user, resume and analysis (`npm run seed`)	seed()
`backend/scripts/migrate-uploads-to-gridfs.js`	One-time move of old uploads into GridFS	—
`backend/tests/*.test.js`	auth (10), google (9), resume (12), analyze (4), europass (13)	describe / it
`backend/tests/helpers.js, setup.js, vitest.config.js`	Test database, mocked Gemini, helpers	—
`.github/workflows/ci.yml`	CI: backend tests + frontend build	jobs: backend, frontend

6.3 How to explain it

"We use MongoDB because the AI feedback and the CV are nested data. There are five collections: users → resumes → analyses → improvedresumes → coverletters. Every document stores `userId`, so queries are always filtered by the owner. Unique indexes make sure there is only one improved CV per analysis. Files are stored in GridFS because Render's disk is wiped on every deploy. Our 48 tests use an in-memory database and a fake Gemini, and GitHub Actions runs them on every push."

Table 10: If sir asks for a change — where to edit

Change request	Edit here	Then check
Add a field to a collection	the model in `backend/src/models/`, then the controller that saves it, `types/index.d.ts`	Tests + the page
Change an input rule (e.g. job description length 20–10 000)	`middleware/validate.js` → `analyzeRules`	analyze.test.js
Change username rules (3–30 chars)	`validate.js` → `registerRules`; form in `AuthCard.tsx`	auth.test.js
Change max edited letter length (8000)	`MAX_EDITED_TEXT` in `models/CoverLetter.js`, `coverLetterEditRules`	europass.test.js
Add a test	new `it(...)` in the matching file in `backend/tests/`	`npm test`
Show demo data	`npm run seed` (uses `database/seed.js`)	Login as demo user

7. Rasel Ratul — Backend Developer

7.1 Responsibilities

- Express 5 REST API: app setup, routes, controllers and central error handling.
- Authentication: register, login with bcrypt + JWT, Google Sign-In verification, `requireAuth` middleware.
- Security: helmet, CORS allow-list, login rate limit, ownership checks, PDF magic-byte check.
- Change password (logs out other devices) and delete account endpoints.
- Resume upload with Multer and the resume endpoints.
- Backend Dockerfile and deployment on Render (`render.yaml`).

Table 11: Technologies used

Technology	Used for
Node.js + Express 5	REST API
bcryptjs	Password hashing
jsonwebtoken	Login tokens (JWT, 7 days)
google-auth-library	Verifies Google ID tokens
Multer	File upload (memory storage, PDF only, 10 MB)
helmet, cors, express-rate-limit	Security headers, allowed origin, brute-force protection
Render, Docker	Production hosting and container image

7.2 Files

Table 12: Files and what they contain

File	What it does	Key functions / parts
`backend/server.js`	Connects to MongoDB and starts the server	—
`backend/src/app.js`	helmet, CORS, JSON parser, routes, error handler	app
`backend/src/config/env.js`	Reads and checks all `.env` settings	config (jwtExpiresIn, maxFileSizeMb, geminiModel …)
`backend/src/routes/authRoutes.js`	Auth URLs + login rate limit (20 / 15 min)	authLimiter
`backend/src/routes/resumeRoutes.js`	Resume and AI URLs	router.get/post/put/delete
`backend/src/controllers/authController.js`	Register, login, Google, me, change password, delete account	register, login, googleLogin, me, changePassword, deleteAccount
`backend/src/controllers/resumeController.js`	Upload, analyze, list, get, file, image, delete	uploadResume, analyze, listResumes, deleteResume
`backend/src/middleware/auth.js`	Checks the JWT, rejects tokens older than a password change	requireAuth
`backend/src/middleware/upload.js`	Multer: PDF/PNG/JPEG only, size limit	uploadResumeFiles
`backend/src/middleware/errorHandler.js`	404 and JSON error responses	notFound, errorHandler

File	What it does	Key functions / parts
backend/src/services/ googleAuthService.js	Verify Google ID token	verifyGoogleToken
backend/src/utils/ token.js, ApiError.js	Sign/verify JWT; error with status code	signToken, verifyToken, ApiError
backend/.env.example	Template of all backend settings	—
backend/Dockerfile, render.yaml	Container image; Render deployment	—

7.3 How to explain it

"When a user logs in, `authController.login()` compares the password with bcrypt and returns a JWT signed by `signToken()`. The browser sends this token with every request. `requireAuth` in `middleware/auth.js` verifies it and loads the user; if the password was changed after the token was issued, the token is rejected. helmet adds security headers, CORS only allows our frontend, and `authLimiter` allows 20 attempts per 15 minutes."

Table 13: If sir asks for a change — where to edit

Change request	Edit here	Then check
Change minimum password length	`validate.js` → `registerRules` and `changePasswordRules` (`min: 8`); forms in `AuthCard.tsx` and `account.tsx`	auth.test.js
Change login attempt limit	`authRoutes.js` → `authLimiter` (`limit: 20`, `windowMs`)	—
Change how long login lasts	`JWT_EXPIRES_IN` in `.env` / Render (default `7d` in env.js)	Log in again
Change max upload size	`MAX_FILE_SIZE_MB` in `.env` / Render + `MAX_FILE_SIZE` in frontend constants.ts	Upload a large PDF
Allow another frontend URL	`CLIENT_URL` on Render (comma-separated)	Open the site
Add a new endpoint	route in `routes/…Routes.js` → function in a controller → rule in validate.js → method in `frontend/app/lib/api.ts` → test	`npm test`
Change what account deletion removes	`authController.deleteAccount()`	auth.test.js

8. Quick Change Lookup

Table 14: Most likely change requests during the viva

If sir asks…	Open this file	Owner
Change password minimum	backend/src/middleware/validate.js + frontend AuthCard.tsx	Ratul / Himel
Change AI prompt or scoring	backend/src/services/geminiService.js → buildPrompt()	Sakib
Change Europass rules	backend/src/services/europassService.js → postCheck()	Sakib
Change cover letter length / tone	backend/src/services/coverLetterService.js	Sakib
Change AI requests per hour	backend/src/routes/resumeRoutes.js → aiLimiter	Sakib
Change login attempts limit	backend/src/routes/authRoutes.js → authLimiter	Ratul
Change token lifetime	JWT_EXPIRES_IN (.env / Render)	Ratul
Change upload size	MAX_FILE_SIZE_MB + frontend/app/constants.ts	Ratul / Mahammad
Add a database field	backend/src/models/<Model>.js + controller + types	Himel
Change validation limits	backend/src/middleware/validate.js	Himel
Add / run a test	backend/tests/*.test.js, `npm test`	Himel
Change colours or fonts	frontend/app/app.css → @theme	Mahammad
Change landing text / team	frontend/app/components/LandingSections.tsx	Mahammad
Add a new page	frontend/app/routes.ts + routes/<page>.tsx	Mahammad
Change score colours	ScoreBadge.tsx, Summary.tsx, ATS.tsx, Details.tsx	Mahammad

9. Making a Change Safely

1. Pull the latest code: `git pull`.
2. Open the file from Chapter 8 and make the change.
3. Run the app locally (`npm run dev` in backend and frontend) and try the feature in the browser.
4. Backend change: run `cd backend && npm test` — all 48 tests must pass. If you changed a rule on purpose (e.g. password length), update the matching test.

5. Frontend change: run `cd frontend && npm run typecheck` and `npm run build`.
6. Commit with a clear message and push: `git add -A`, `git commit -m "…"`, `git push`.
7. GitHub Actions runs the tests; Vercel and Render redeploy automatically in a few minutes.
8. If you changed a `VITE_…` variable on Vercel, click **Redeploy** there; if you changed a variable on Render, it redeploys by itself.

9.1 Settings that are changed without code

Variable	Where	Effect
`GEMINI_MODEL`	backend .env / Render	Which Gemini model is used
`JWT_EXPIRES_IN`	backend .env / Render	How long a login lasts (default 7d)
`MAX_FILE_SIZE_MB`	backend .env / Render	Upload limit in MB (default 10)
`CLIENT_URL`	backend .env / Render	Frontend address allowed by CORS
`GEMINI_API_KEY`, `MONGO_URI`, `JWT_SECRET`	backend .env / Render	Secrets — never commit them
`VITE_API_URL`	frontend .env / Vercel	Backend address used by the frontend
`VITE_GOOGLE_CLIENT_ID`	frontend .env / Vercel	Shows the Google Sign-In button